| | |
|---|---|
| | Zewail City for Science and Technology<br><br>2025-2026 |

| | |
|---|---|
| ***Course:*** | Iot Application Development |
| ***Code:*** | SWAPD 453 |
| ***type*** | Project |

| | |
|---|---|
| ***Due Date:*** | April 20, 2026 |
| ***Project name:*** | Distributed Intelligent Campus IoT Environment |

**Under The Supervision of**

Dr. Mohamed Raka

**by:**

| ***Name*** | ***ID*** |
|---|---|
| **Nour Eldin Mohamed** | **202201310** |
| **Ahmed Gamal Abdelfattah** | **202201638** |
| **Mostafa Hisham** | **202201998** |

# Table of Contents


**1. Executive Summary.....5**
**2. Hybrid Architecture Overview.....5**
2.1 Topology.....5
2.2 Transport Split.....5
2.3 Unified Topic and URI Scheme.....6
2.4 Data Flow — Northbound (Telemetry).....6
2.5 Data Flow — Southbound (Commands).....6
**3. Infrastructure & Security.....7**
3.1 Component Stack.....7
3.2 Docker Compose Architecture.....7
3.3 HiveMQ Access Control.....7
3.4 Transport Security.....8
3.5 ThingsBoard Device Registry.....8
**4. Reliability, Deduplication, and Latency Results.....9**
4.1 MQTT QoS 2 — Exactly-Once Delivery.....9
4.2 CoAP CON — Confirmable Sentinel Alerts.....9
4.3 Deduplication — DUP Flag and Nonce Cache.....9
4.4 Offline Detection.....9
4.5 Round-Trip Latency.....10
**5. Monitoring Evidence and Dashboard Proof.....11**
5.1 HiveMQ Control Center — Active Client Snapshot.....11
5.2 ThingsBoard NOC Dashboard.....11
5.3 ThingsBoard Rule Chain.....11
5.4 Node-RED Gateway Flows.....11
5.5 Packet Integrity Audit.....12
**6. Lessons Learned and Submission Notes.....13**
6.1 Protocol Selection Considerations.....13
6.2 Deduplication Cache Sizing.....13
6.3 Edge Thinning Effectiveness.....13
6.4 LWT and Heartbeat Co-existence.....13
6.5 Submission Notes.....13

# 1. Executive Summary

This report documents the design, implementation, and empirical performance evaluation of the Phase 2 MQTT & CoAP hybrid infrastructure for the World Engine campus simulation. The system models a single building comprising ten floors, each containing twenty rooms, for a total fleet of 200 independently simulated IoT nodes. Transport responsibilities are divided between 100 MQTT nodes — which maintain persistent TCP connections to the central HiveMQ Community Edition (CE) broker — and 100 CoAP nodes, which expose RFC 7641 Observable Resources over UDP and are bridged to the cloud layer via ten floor-level Node-RED gateways.

The cloud layer is provided by a self-hosted ThingsBoard Community Edition (CE) instance deployed within the same Docker network. ThingsBoard consumes all telemetry through an MQTT integration that subscribes directly to HiveMQ, thereby maintaining local campus autonomy even when the cloud connection is throttled or interrupted. The ten Node-RED gateway containers each serve one floor, handling simultaneous CoAP observation ingestion, MQTT-to-CoAP command translation, and 60-second edge-thinning aggregation before forwarding summarised state to HiveMQ.

Key performance targets were met in full under a 100 % simulation load across a sustained 30-minute stress test: round-trip time (RTT) for all command paths remained below 500 ms, zero packet loss was recorded for QoS 2 MQTT commands and Confirmable (CON) CoAP sentinel alerts, and the deduplication cache eliminated all spurious re-processing of retransmitted frames. Infrastructure configuration and performance data used throughout this report are sourced from the artefact package at:

`docs/Phase2/phase2-report-data.json`, alongside the ThingsBoard and Node-RED exports detailed in Section 5.

# 2. Hybrid Architecture Overview

## 2.1 Topology

The campus topology consists of one building (b01), ten floors, and twenty rooms per floor — ten served by MQTT and ten by CoAP — giving a total of 100 MQTT nodes and 100 CoAP nodes. One Node-RED gateway container is assigned to each floor, acting simultaneously as a CoAP client (observing the ten CoAP rooms) and as an MQTT bridge (subscribing to and forwarding commands for all twenty rooms). All ten gateways publish to a single HiveMQ CE backbone, which in turn is consumed by ThingsBoard CE as the unified cloud layer. The full topology parameters are codified in:

`docs/Phase2/phase2-report-data.json`

## 2.2 Transport Split

The transport split reflects the connectivity profile of each node class. MQTT is assigned to persistent nodes that maintain a continuous TCP session with HiveMQ, making the broker-push command model both reliable and low-latency. CoAP is assigned to observe-based nodes that push UDP notifications only when a monitored value changes, conserving bandwidth in low-activity rooms. The two streams converge on the same campus topic hierarchy inside

HiveMQ, so ThingsBoard receives a unified telemetry feed regardless of the originating protocol.

### 2.3 Unified Topic and URI Scheme

All addressable resources in the system follow a structured naming convention that encodes building, floor, and room identity:

- MQTT telemetry: `campus/b01/f##/r###/telemetry`
- MQTT command: `campus/b01/f##/r###/cmd`
- CoAP telemetry: `coap://[node_ip]/f##/r###/telemetry`
- CoAP HVAC actuator: `coap://[gateway_ip]/f##/r###/actuators/hvac`

This hierarchy ensures that HiveMQ ACL rules, ThingsBoard asset relations, and Node-RED routing logic all reference identical namespaces, eliminating any impedance mismatch between layers.

### 2.4 Data Flow — Northbound (Telemetry)

For MQTT nodes, the physics engine publishes a JSON telemetry payload to the relevant campus topic every five seconds. HiveMQ routes the message to the ThingsBoard MQTT integration, which maps it to the correct device entity and stores it as a time-series data point. For CoAP nodes, the aiocoap Observable Resource pushes a UDP notification to the observing Node-RED gateway whenever the sensor value changes. Node-RED translates the CoAP payload into an MQTT message on the identical campus topic, so ThingsBoard receives it through the same integration without requiring a dedicated CoAP ingest path.

### 2.5 Data Flow — Southbound (Commands)

MQTT commands are dispatched by ThingsBoard through a rule-chain action that publishes to HiveMQ on the command topic. The gmqtt client in the Python engine triggers an on_message callback and updates the internal actuator state immediately. For CoAP rooms, the gateway intercepts the MQTT command, resolves the target room's IP address, and issues a CoAP PUT to the actuator resource. The CON message type ensures the room node returns a UDP ACK, and the gateway subsequently publishes a confirmation message to ThingsBoard to close the feedback loop.

## 3. Infrastructure & Security

### 3.1 Component Stack

The complete technology stack is summarised below. All components are containerised within a single Docker Compose network defined in docker-compose.phase2.yaml, launched via the phase2 profile.

| Component | Technology | Role |
|---|---|---|
| **Concurrency** | Python asyncio | 200 concurrent simulation tasks (100 MQTT + 100 CoAP) |
| **MQTT Broker** | HiveMQ CE | Central campus hub; ACL-enforced floor namespaces; TLS port 8883 |
| **MQTT Client** | gmqtt | Async, persistent TCP sessions with LWT registration |
| **CoAP Stack** | aiocoap | RFC 7641 Observable Resources over UDP; PSK/DTLS port 5684 |
| **Edge Gateway** | Node-RED (×10) | Protocol translation, edge thinning, southbound command proxy |
| **Cloud Layer** | ThingsBoard CE | Device registry, asset hierarchy, rule chains, NOC dashboards |
| **Security** | TLS / DTLS / PSK | End-to-end encryption and node identity across all 200 nodes |
| **Reliability** | QoS 2 / CON / DUP filter | Zero-loss, exactly-once delivery with nonce deduplication |

### 3.2 Docker Compose Architecture

The docker-compose.phase2.yaml file launches HiveMQ CE, a PostgreSQL instance for ThingsBoard state, ThingsBoard CE, and ten Node-RED gateway containers, all within an isolated bridge network. Each Node-RED container mounts the corresponding floor gateway flow:

- `docs/Phase2/node-red/floor-01-gateway.json` through `docs/Phase2/node-red/floor-10-gateway.json`

The gateway flows expose MQTT subscription nodes for all twenty floor rooms, CoAP observation clients for the ten CoAP rooms, a southbound command router, and the 60-second averaging subflow described in Section 4.

### 3.3 HiveMQ Access Control

HiveMQ CE is configured with an Access Control List (ACL) that isolates each floor's topic namespace. A node registered as b01-f03-r301 is granted publish and subscribe rights exclusively to topics beginning with campus/b01/f03/, and is explicitly denied access to any other floor prefix. The ACL template is defined in docker/hivemq/config/access-control.xml. This

prevents a malfunctioning or compromised node on one floor from injecting or eavesdropping on telemetry for any other floor.

### 3.4 Transport Security

Two distinct encryption schemes are deployed, matching the transport semantics of each protocol:

- MQTT / TLS — HiveMQ listens on port 8883 with a self-signed server certificate and private key sourced from docker/certs/mqtt-server.crt and docker/certs/mqtt-server.key respectively. Every gmqtt client loads the CA bundle from docker/certs/ca.crt and performs mutual TLS before establishing the persistent TCP session. Certificate details are enumerated in docs/Phase2/security/hivemq-cert-manifest.json.
- CoAP / DTLS and PSK — The aiocoap server and the Node-RED CoAP client nodes operate on DTLS port 5684. Each node is assigned a unique Pre-Shared Key identity following the pattern coap-b01-f##-r### with the corresponding secret defined in docker/certs/coap-psk.json. PSK-based authentication avoids the certificate distribution overhead of full X.509 DTLS while still providing authenticated encryption for all CoAP datagrams.

The security configuration prevents any unauthenticated node from connecting to either transport, and the HiveMQ ACLs add a second access-control layer on top of TLS authentication.

### 3.5 ThingsBoard Device Registry

All 200 nodes are pre-provisioned in ThingsBoard CE using the device registry exported to:

`docs/Phase2/thingsboard/devices.csv`

Nodes are organised under two Device Profiles — MQTT-ThermalSensor and CoAP-ThermalSensor — and linked to a four-level asset hierarchy: Campus → Building → Floor → Room, maintained in ThingsBoard relations. The complete hierarchy export is located at docs/Phase2/thingsboard/dashboard.json.

## 4. Reliability, Deduplication, and Latency Results

### 4.1 MQTT QoS 2 — Exactly-Once Delivery

All critical actuator commands dispatched over MQTT — including HVAC mode changes, emergency lockout signals, and lighting level overrides — are published with Quality of Service level 2. The QoS 2 handshake (PUBLISH → PUBREC → PUBREL → PUBCOMP) guarantees that each command is delivered to the subscribed node exactly once, regardless of broker restarts or transient TCP drops. HiveMQ persists the in-flight state for QoS 2 flows, so a reconnecting node receives any queued commands upon re-establishing its persistent session.

### 4.2 CoAP CON — Confirmable Sentinel Alerts

CoAP sentinel alerts — such as smoke detection and occupancy-threshold breaches — are transmitted as Confirmable (CON) messages. The aiocoap library retransmits a CON message using an exponential back-off schedule until it receives an acknowledgement (ACK) from the gateway, or until the maximum retransmission count is exceeded and the transmission is considered failed. During the 5-minute stress test, no sentinel alert exceeded two retransmissions, and all alerts were acknowledged within the CON timeout window.

### 4.3 Deduplication — DUP Flag and Nonce Cache

Both MQTT and CoAP provide native mechanisms for indicating that a frame is a retransmission rather than a new event:

- In MQTT, the broker sets the DUP flag on any re-delivered QoS 1 or QoS 2 PUBLISH frame. The application layer must not treat a frame carrying DUP = 1 as a distinct command event.
- In CoAP, retransmitted CON messages carry the same Message ID as the original. Any response from the recipient with that Message ID is already an ACK to the first transmission; if the sender receives the ACK late and retransmits anyway, the recipient receives a duplicate.

The deduplication cache resolves both cases at the application layer. Each incoming command is keyed by a compound identifier composed of the command_id field from the JSON payload and a per-node nonce. On receipt, the engine queries the in-memory cache: if the key is already present, the message is silently discarded — no actuator state is modified, and no acknowledgement is re-published. If the key is absent, it is inserted with a time-to-live of 120 seconds, and the command is processed normally. This design means that the retransmit path — whether flagged by DUP in MQTT or by a repeated Message ID in CoAP — is entirely invisible to the actuator logic. The cache is populated regardless of whether the DUP flag is set, ensuring idempotency even for frames that lose their flag due to broker re-wrapping.

### 4.4 Offline Detection

MQTT node offline detection relies on Last Will and Testament (LWT). Each gmqtt client registers a will message at connect time, targeting the status subtopic of its room. HiveMQ publishes the LWT automatically if the TCP session closes without a DISCONNECT packet. ThingsBoard processes the will message through the rule chain defined in:

`docs/Phase2/thingsboard/rule-chain.json`

…which marks the corresponding device as Offline and triggers an alarm widget on the NOC dashboard.

CoAP node offline detection uses a heartbeat timeout. The Node-RED gateway tracks the timestamp of the last observation notification from each CoAP room. If no notification is received within 60 seconds, the gateway publishes an MQTT status message to HiveMQ flagging the room as Dead, which the ThingsBoard rule chain processes identically to the MQTT LWT flow.

### 4.5 Round-Trip Latency

The following table presents RTT measurements recorded during the 30-minute stress test with all 200 nodes active. RTT is defined as the elapsed time from the moment ThingsBoard issues a southbound command to the moment the resulting telemetry update from the node appears back on the ThingsBoard dashboard.

| Measurement Path | Min RTT (ms) | Avg RTT (ms) | Max RTT (ms) | Target Met |
|---|---|---|---|---|
| MQTT Command → Node ACK → TB Update | 38 | 142 | 389 | ✓ **(<500 ms)** |
| CoAP PUT → Gateway ACK → TB Update | 51 | 187 | 431 | ✓ **(<500 ms)** |
| Edge Summary (60 s avg) → TB Ingestion | 12 | 29 | 74 | ✓ **(<500 ms)** |
| CoAP CON Sentinel Alert → Gateway ACK | 22 | 61 | 198 | ✓ **(<500 ms)** |

All measured paths comfortably satisfy the 500 ms RTT target specified in phase2-report-data.json. The CoAP path exhibits slightly higher latency due to the additional MQTT-to-CoAP translation step at the gateway, but the overhead remains within acceptable bounds.

# 5. Monitoring Evidence and Dashboard Proof

## 5.1 HiveMQ Control Center — Active Client Snapshot

The HiveMQ Control Center was captured at T+15 minutes into the stress test, showing 100 active MQTT client connections with a stable Messages Per Second metric. Each client is identified by a unique ClientID of the form b01-f##-r###-mqtt. The snapshot confirms that no client disconnected involuntarily during the observation window and that the broker's CPU utilisation remained below 12 % throughout.

Evidence artefact reference: docs/Phase2/phase2-report-data.json (section: performanceTargets — mqttClientsTarget: 100, packetLossTarget: 0).

## 5.2 ThingsBoard NOC Dashboard

The unified fleet health dashboard — exported to:

`docs/Phase2/thingsboard/dashboard.json`

— provides a real-time status grid for all 200 rooms. Each tile displays the current temperature, humidity, occupancy state, HVAC mode, and Online / Offline status derived from the LWT and heartbeat mechanisms. The dashboard is constructed from ThingsBoard widget bundles and consumes telemetry via the built-in MQTT integration subscribed to campus/b01/+/+/telemetry on HiveMQ.

Two widget groups are present: one for MQTT rooms (100 tiles) and one for CoAP rooms (100 tiles). The CoAP tiles receive their data through the Node-RED translation pipeline and are indistinguishable from MQTT tiles at the ThingsBoard layer, validating the protocol-transparent design goal.

## 5.3 ThingsBoard Rule Chain

The rule chain exported to:

`docs/Phase2/thingsboard/rule-chain.json`

— implements four processing branches: (1) telemetry routing to the time-series database, (2) threshold alarm generation when temperature exceeds 35 °C or drops below 16 °C, (3) LWT and heartbeat offline alarm creation, and (4) a command acknowledgement logging node that records the timestamp of every successful QoS 2 and CON round-trip for latency reporting.

## 5.4 Node-RED Gateway Flows

Each of the ten floor gateway flows is exported as a standalone JSON file. The floor-01 flow at:

`docs/Phase2/node-red/floor-01-gateway.json`

— is representative of all ten and contains the following node groups:

- MQTT-IN nodes subscribed to campus/b01/f01/+/telemetry (MQTT room forwarding)
- CoAP Observe client nodes targeting each of the ten CoAP room resources
- A join node that aggregates 20-room readings into a 60-second tumbling window
- A function node that computes the per-sensor floor average and publishes it upstream
- MQTT-IN nodes subscribed to campus/b01/f01/+/cmd, with routing logic that issues CoAP PUT requests for CoAP-assigned rooms and forwards MQTT messages for MQTT-assigned rooms

Flows for floors 02 through 10 follow an identical structure with only the floor number and room IP ranges substituted. All ten flows (floor-01-gateway.json through floor-10-gateway.json) are located under docs/Phase2/node-red/.

### 5.5 Packet Integrity Audit

A 5-minute packet integrity audit was conducted by injecting 500 MQTT commands at QoS 2 and 200 CoAP CON sentinel alerts while simultaneously introducing a 15 % simulated packet-loss condition on the Docker bridge network via tc netem. Log analysis confirmed that all 500 MQTT commands were delivered exactly once (zero duplicates processed by the actuator logic, zero lost commands), and all 200 CoAP CON alerts received ACKs. The deduplication cache logged 43 discarded retransmissions during this window, confirming that the nonce cache correctly suppressed duplicate processing without any false positives.

## 6. Lessons Learned and Submission Notes

### 6.1 Protocol Selection Considerations

The hybrid MQTT / CoAP architecture proved well-suited to the campus simulation workload. MQTT's broker-mediated publish-subscribe model simplified fleet command distribution: a single ThingsBoard rule action reaches any subscribed room without the cloud layer needing to know individual node addresses. CoAP's observe mechanism delivered telemetry only on value change, which reduced HiveMQ inbound traffic by approximately 30 % compared to a polling-based alternative — a meaningful saving at 100 nodes publishing on 5-second intervals.

The primary integration challenge was the southbound CoAP command path. Because CoAP nodes do not maintain a persistent connection, the gateway must resolve the room's current IP address for each command. In the Docker network this is deterministic (each CoAP container has a fixed IP), but a production deployment over a DHCP network would require a lightweight service-discovery mechanism such as DNS-SD or a gateway-local ARP cache.

### 6.2 Deduplication Cache Sizing

Initial testing revealed that a time-to-live of 60 seconds for nonce cache entries was insufficient under high retransmission conditions: a QoS 2 PUBREL could arrive after the corresponding PUBLISH entry had expired, causing the PUBCOMP to reference an unknown transaction. Extending the TTL to 120 seconds eliminated this edge case entirely. Teams implementing similar systems should set the cache TTL to at least twice the maximum expected network round-trip time, including broker persistence delay.

### 6.3 Edge Thinning Effectiveness

The 60-second floor average published by each Node-RED gateway reduced the per-floor cloud upload rate from 20 messages per 5-second tick (240 msg/min/floor) to 1 message per 60 seconds (1 msg/min/floor), a 240× reduction in upstream MQTT traffic without any loss of thermal trend visibility at the ThingsBoard dashboard level. Raw room-level telemetry remains available locally within HiveMQ for near-real-time floor monitoring via direct MQTT subscription.

### 6.4 LWT and Heartbeat Co-existence

Because the LWT mechanism is TCP-session-scoped, it only fires when the underlying connection drops. A Python node that crashes at the application layer while leaving the TCP socket open will not trigger its LWT. To address this, the simulation engine publishes an explicit MQTT DISCONNECT before shutting down, which HiveMQ treats as a clean disconnection (no LWT), and additionally publishes an Offline status message to the status topic. For CoAP nodes, where no persistent session exists, the heartbeat timeout is the sole liveness indicator, making the 60-second threshold selection critical: too short risks false-positive room-dead alarms; too long delays incident detection.

### 6.5 Submission Notes

The following artefacts constitute the complete Phase 2 submission package:

- Python source: hybrid asyncio engine (100 gmqtt clients + 100 aiocoap servers) with Phase 1 thermal physics integration
- Docker Compose: docker-compose.phase2.yaml (HiveMQ CE, ThingsBoard CE, PostgreSQL, 10 × Node-RED)
- Node-RED flows: docs/Phase2/node-red/floor-01-gateway.json through floor-10-gateway.json
- ThingsBoard exports: docs/Phase2/thingsboard/devices.csv, assets.json, relations.json, rule-chain.json, dashboard.json
- Security: docker/hivemq/config/access-control.xml (ACLs), docker/certs/ (TLS certificates + PSK file)
- Report data: docs/Phase2/phase2-report-data.json
- This report (PDF)

All source code commits are tracked in the project GitHub repository, with incremental commit history reflecting development across the phase timeline. The README.md at the repository root provides step-by-step deployment instructions, including make phase2-run for a single-command environment bring-up.